RZ V5 – Synthèse échange des données – Structure Arduino

Table des matières

SYSTEME & ARCHITECTURE SUPÉRIEURE....5
Architecture générale SE ↔ SP ↔ MQTT ↔ Arduino....5
Rôle des appareils....5
◆ Matériel....5
Architecture réseau....5
Contraintes : ressources et temporelles (urgent / normal / lent)....5
TOPICS MQTT....6
Généralité sur l'échange de données MQTT / Broker et Topics....6
Deux lignes d'échanges : SE ↔ SP et Ardu ↔ SP....6
Structure topics ardu ↔ Node-Red....7
Structure topics *Tasker* ↔ *Node-Red* noté SE ↔ Node-RED....7
stratégie QoS en fonction de l'"urgence"....7
PROTOCOLE DE DONNÉES – Structure : $<cat>:<var>:<prop>:<inst>:<val>#....8
Catégorie - Usage réel et final des catégories VPIV (V / I / E / F)....9
Bien choisir la catégories....9
V/I/E = maître / esclave / fiabilité....9
F = streaming temps réel + non bloquant....9
Cas de F pour Flux haute fréquence et UNIQUEMENT flux haute fréquence....9
Cas de V – Catégorie utilisé pour les fonctions ….....10
Variables Stockage, abréviations & propriétés....10
Stockage Node-RED — 2 niveaux....10
A) runtime variables (global/flow/local) — pour traitement temps réel....10
B) fichier variables.json — source de vérité pour initialisation + documentation....10
Règles :....10
Synchronisation initiale (P3.3 / P3.4)....10
Types & mapping....11
INSTANCE....11
Modules arduino....12
Architecture interne : "Modules indépendants + cœur système"....12
MODULE....12
Transfert d'info entre module....12
Transfert par lecture directe de fonctions (pull)....12
Transfert par buffer partagé (push)....13
Message interne (événement)....13
Interaction complète : "commande externe → action interne → flux interne → info externe"....14
Interaction interne complexe : "traitements secondaires"....14
Où passent les paramètres "complexes / secondaires" ?....15
Exemple :....15
Propriété....16
4 VALEUR....16
Exemples de messages....16
Gestion des erreurs VPIV....17

Gestion des urgences....17
Communication :....17
7. Flux périodique vs commandes....18
8. Sécurité du protocole....18
Robustesse garantie par :....18
Pourquoi c'est mieux que l'ancien système tablette ?....18
Ancien :....18
Nouveau VPIV :....18
MODULE PAR MODULE (DOCUMENTATION COMPLÈTE)....19
l r u b (ruban LED RGB)....19
1. Rôle....19
2. Variables VPIV....19
Détail des propriétés :....19
3. Structure interne....19
4. Exemples VPIV....19
Allumer en rouge :....19
Baisser l'intensité :....19
Animation n°3 :....19
5. Exemple de réponse....19
6. Limites....19
l r i n g (anneau LED expressif)....20
1. Rôle....20
2. Variables VPIV....20
Propriétés :....20
3.Structure interne....20
4. Exemples VPIV....20
5. Réponse :....20
6. Limites....20
S r v (servos tête / bouche)....21
1. Rôle....21
2. Variables VPIV....21
Propriétés :....21
3. Structure interne....21
4. Exemples VPIV....21
5. Réponse....21
6. Limites....21
Mt (moteurs via Arduino UNO)....22
1. Rôle....22
2. Variables VPIV....22
Propriétés :....22
3. Structure interne....22
4. Exemples VPIV....22
5. Réponse....22
6. Limites....22
U S (sonars ultrason HC-SR04)....23
1. Rôle....23
2. Variables VPIV....23
3. Structure interne....23
4. Exemples VPIV....23
5. Réponse....23

6. Limites..... 23
Mic (array de microphones + direction)..... 24
1. Rôle du module..... 24
2. Variables VPIV..... 24
Détails des propriétés..... 24
3. Structure interne..... 24
4. Exemples VPIV..... 24
5. Réponses..... 24
6. Limites..... 24
Odom (odométrie encodeurs)..... 25
1. Rôle..... 25
2. Variables VPIV..... 25
3. Structure interne..... 25
Mise à jour des ticks :..... 25
Distance :..... 25
Orientation :..... 25
Position :..... 25
4. Exemples VPIV..... 25
5. Réponses..... 25
6. Limites..... 25
Mvt_ir (détecteur infrarouge PIR)..... 26
1. Rôle..... 26
2. Variables VPIV..... 26
Propriétés..... 26
3. Structure interne..... 26
4. Exemples VPIV..... 26
5. Réponse..... 26
6. Notes..... 26
FS (capteur de force HX711)..... 27
1. Rôle..... 27
2. Variables VPIV..... 27
Propriétés..... 27
3. Structure interne..... 27
4. Exemples VPIV..... 27
5. Réponses..... 27
6. Limites..... 27
SCfg (Système / Réglages / Emergency)..... 28
1. Rôle général du module SCfg..... 28
2. Variables VPIV..... 28
Détails :..... 28
3. Rôle dans la chaîne d'urgence..... 28
4. Exemples VPIV..... 28
5. Réponses..... 29
6. Notes..... 29
✅ TABLEAU GLOBAL VPIV — Tous les modules (prêt pour Node-RED)..... 30
🟦 1 — Module LRUB (Ruban LED)..... 30
Constantes importantes (config.h)..... 30
🟦 2 — Module LRING (Anneau LED expressif)..... 30
Constantes..... 30
🟦 3 — Module SERVOS..... 30

Constantes..... 31
4 — Module MOTEURS (UNO)..... 31
Constantes..... 31
5 — Module ULTRASON (US)..... 31
Constantes..... 31
6 — Module MICROPHONES (Mic)..... 32
Constantes..... 32
7 — Module ODOMÉTRIE (Odom)..... 32
Constantes..... 32
8 — Module IR (MvtIR)..... 32
Constantes..... 33
9 — Module FS (Capteur de force)..... 33
Constantes..... 33
10 — Module SYSTÈME (SCfg)..... 33
Constantes..... 33
RÉCAP POUR NODE-RED (clé → valeur)..... 34
Exemple structure d'init Node-RED :..... 34
VI — ARCHITECTURE DE TEST DU ROBOT..... 35
----------..... 35
1 — Types de tests intégrés..... 35
2 — Format des rapports..... 35
3 — Tests automatisés en Node-RED..... 35
PERSPECTIVES ET EXTENSIONS - A FAIRE..... 36
----------..... 36
1 — Ajout d'une boussole externe (I2C QMC5883L)..... 36
2 — Fusion capteurs pour autonomie..... 36
3 — Calibration assistée via VPIV..... 36
4 — Logs experts..... 36
ANNEXES..... 37

SYSTEME & ARCHITECTURE SUPÉRIEURE

Architecture générale SE ↔ SP ↔ MQTT ↔ Arduino

📘 Deux lignes distinctes pour les échanges DATA :

- **Echange avec les composants électroniques externes** : Arduino mega → USB → Smartphone SE → Arduino IoT UART Gateway → MQTT-→ Pilote (SP ou autre) → Node-RED .
  Appli Arduino IoT UART Gateway : Configuration du Bridge bidirectionnel (usb +MQTT)
- **Pour les échanges avec les « outils » de SE** : SE → Tasker → MQTT + autres → Pilote dont SP. Accès aux capteurs, actionneurs et manipulation d'appli à distance.

  Pas de référence ici aux échanges SSH, VNC,…

Rôle des appareils

🔶 Matériel

- **Arduino MEGA** - maître de bas niveau : moteurs, capteurs, sécurité) → porte électronique externe
- **Smartphone SE** - Ordinateur embarqué :cerveau robot → connecté en USB à Arduino + porte applis android et source de capteurs et actionneurs (inclus camera, écran, micros, ...)
- **Arduino IoT UART Gateway** - app bridge Arduino ↔ USB ↔ MQTT
- **RPI3 (serveur)** → Broker Mosquitto + Serveur web + Node-RED (clone de celui de SP)
- **Smartphone SP** → Pilotage, interface Node-RED Dashboard + gestion globale des variables du système (inclus un fichier stable des variables globales). : **intermédiaire MQTT -orchestrateur des commandes - dashboard interactif - logique additionnelle (mode auto → sérialisation des ordres)- contrôle de sécurité - journalisation longue durée...**
- **Autres pilotes** → Le pilotage peux se faire par tout appareil ayant une liaison avec le RPI serveur

  Nota bene : SE, SP et RPI3 ont tous linux (ou termux)

Architecture réseau

- Le SE **n'a pas de SIM**
- Le SP crée un **Hotspot**
  → SE + ESP éventuel + SP se connectent dessus
- Le RPI3serveur est accessible via **192.168.1.40** en Wi-Fi local
- Un accès extérieur existe via DuckDNS (fournisseur de nom de domaine)

Contraintes : ressources et temporelles (urgent / normal / lent)

La gestion des échanges de données entre un robot et son pilote dans un contexte de ressources limitées nécessite des adaptations. Les choix retenus :

- Choix de l'abandon du bluetooth pour les échanges de données → simplification, fiabilisation
- Un seul pilote à la fois → simplification, fiabilisation
- une variable qui structure l'interprétation des données, modifie le volume d'échanges : le mode de fonctionnement, cette variable défini le mode du robot : autonome, veille, arrêt, déplacement,..
- Utilisation de MQTT comme protocole pour le pilotage des interactions entre les appareils
- Mise en cohérence des variables (nom, structure,…) dans toute l'application
- Gestion des topics hiérarchique avec en niveau 1 l'appareil source → ressources, gestion temporelle
- Encodage des données d'échanges → VPIV
- Message de taille limité (inférieur à 100 caractères)
- Intégration de la contrainte temporelle dans les deux lignes d'échanges :
  - → Vers Tasker : topics hiérarchique avec ref sur l'urgence (aide classement QoS)
  - → Vers Arduino : inclusion de ref sur l'urgence dans la chaine de donnée transmise (adaptation à la réception coté Node-Red, intrinsèque au type d'information coté Arduino

---

TOPICS MQTT

Généralité sur l'échange de données MQTT / Broker et Topics

Tout le trafic MQTT est sous le broker installé sur RPI3 serveur . La création d'un broker sur SE n'a pas été retenu. La configuration du serveur MQTT et des clients est développé sur un document séparé.

Deux lignes d'échanges : SE ↔ SP et Ardu ↔ SP

Tasker (SE) ↔ Node-Red (SP) – On considère ici, bidirectionnel SE ↔ SP

Arduino ↔ IotGateway (SE) ↔ Node-Red (SP) On considère ici bidirectionnel Ardu ↔ SP

Des contraintes « différentes » suivant l'application de traitement :

Tasker sait gérer du Json propre, mais c'est lourd à réaliser (alternative utiliser un intermédiaire mosquitto via termux). Les échanges mqtt entre Tasker et Node-RED sont moins nombreux et nettement plus faible fréquence. Pour info tasker est fort pour les Regex (Expression Régulière) et tasker gère bien les sous partie de topic. Q3 je ne fait circuler que des petits messages nettement inférieur à 100 caractères.

Le choix retenu sur la structure des échanges pour chaque ligne

Modèle unifié Arduino & SE

On retiens un protocole compact pour tasker et on calque sur un minimum de topic, ce qui n'était pas le choix initial. La seul différence restante entre le modèle des deux lignes est qu'il existe dans le schéma vpiv des catégories supplémentaires vers tasker : app (application) even (événement).

👉 Résultat : cohérence d'encodage, `$V:xxx:yyy:...#` est trivial. Les messages SE sont légers, donc **protocole compact = idéal**.
SE ↔ SP utilise le même cœur que Arduino ↔ SP,
mais avec quelques extensions spécifiques à SE (applications, événements…).

Structure topics ardu ↔ Node-Red

Limité à trois par l'appli IOT Gateway

- `arduino/commande` (SP → Arduino)
- `arduino/reponse` (Arduino → SP)
- `arduino/statut` (Arduino → SP)
- SE (smartphone embarqué) ↔ SP (smartphone pilote/Node-RED) :

Structure topics *Tasker ↔ Node-Red* noté SE ↔ Node-RED

Pas de limite par l'appli, mais on limiter si possible à :

- `SE/commande` (SP → SE)
- `SE/reponse` (SE → SP)
- `SE/statut` (SE → SP)

si besoin on complexifiera : `SE/<cat>/<urg>/<var>` (payload = `$...#`) avec cat de type : V, I, F, E, A (appli en plus…). Mais ceci pour faciliter traitement coté Tasker, coté Node-Red on utilisera le même traitement (lecture de la chaine de donnée, malgré redondance d'info avec le topic de reception).

Remarque : pas événement dans les catégorie au final, car le déclencheur sera le traitement de la recption d'un topic, les événemnt coté Tasker venant de l'extérieur (capteurs,…) déclencheront eux des topics structuré pour node-red (utile en mode autonome)

stratégie QoS en fonction de l'"urgence"

Associe les urgences à QoS (recommandation pratique) :

- `urgent` → QoS 2 (livraison garantie, rare et court messages — ok)
- `normal` → QoS 1
- `lent` → QoS 0

Remarques :

- QoS2 crée surcharge côté broker ; réserver `urgent` aux événements critiques (erreur, sécurité).
- Pour flux très fréquent (microphone), utiliser `normal` (QoS1) ou `lent`(QoS0) selon tolérance.

PROTOCOLE DE DONNÉES – Structure : `$<cat>:<var>:<prop>:<inst>:<val>#`

- Catégorie (V, I, F, E, A) : valeur, info, fonction, erreur, application (Pour événement?)
- Variable : abbrVarG (ex `DVar`), ou un nom court côté SE (`Lumi`, `App`).
- Propriété : abbrP (`val`, `state`, `nbrev`, `act`...)
- Instance : `*` = default ou `inst1` etc. Pour SE, `inst` restera `0` ou `*` (pas d'instances multiples).
- Valeur : chaîne codée suivant `typeP` (voir section mapping types).

Élément	Nom	Exemple	Rôle
`$`	début	`$V:…`	commence le message
`<CAT>`	catégorie	`V`, `I`, `E`, `F`	voir tableau ci-dessous
`<MODULE>`	module ciblé	`US`, `Mic`, `Odom`	identifie le composant
`<INSTANCE>`	instance	`*`, `0`, `[1,3]`	s'applique à plusieurs capteurs
`<PROPRIÉTÉ>`	action / variable	`read`, `freq`, `raw`	commande
`<VALEUR>`	donnée numérique / chaîne	`1`, `350`, `red,10`	paramètre
#	fin de trame		protège le protocole

Exemples :

- Mise à jour valeur luminosité :
  Topic: `SE/V/normal/DVar`
  Payload: `$V:DVar:val:*:127#`
- Lancement d'application (catégorie A) :
  Topic: `SP/A/normal/App`
  Payload: `$F:App:launch:*:chrome#`
- Etat d'une appli SE :
  Topic: `SE/A/normal/App`
  Payload: `$I:App:state:*:youtube#`

Catégorie - Usage réel et final des catégories VPIV (V / I / E / F)

Catégorie	Sens officiel	Direction	Rôle exact
V	*Commande (action ou lecture)*	Arduino → Node-RED	Demande explicite : modifier un état, lire une valeur, activer un module.
I	*Information / réponse*	Arduino → Node-RED	Réponse à une commande V **ou** info spontanée (ex : « servo prêt », « reboot »).
E	*Erreur*	Arduino → Node-RED	Signalement d'un problème (timeout, value invalid…).
F	*Flux*	Arduino → Node-RED	Messages envoyés **à haute fréquence**, typiquement capteurs ou odométrie.

Bien choisir la catégories

V/I/E = maître / esclave / fiabilité

- V = intention claire
- I = réponse fiable, synchrone
- E = gestion d'erreur robuste
  Ces messages supportent la logique métier.

F = streaming temps réel + non bloquant

- pas de handshake
- pas d'attente
- traitement léger
- aucune garantie d'ordre
- aucune tentative de réémission

→ C'est ce qui permet d'éviter de saturer Arduino ou Node-RED quand un module envoie 20–100 messages/s.

Cas de F pour Flux haute fréquence et UNIQUEMENT flux haute fréquence

F n'est PAS un message de configuration.
Il sert exclusivement aux modules qui doivent émettre en continu :

- Odométrie (positions / vitesses)

- Ultrasons (si mode scan haute fréquence)
- Microphones (intensité / direction)
- IR (mouvements rapides)
- Logs de débogage temps réel

✔ F est traité **dans une pipeline optimisée** (sans acquittement, sans latence).
✔ Node-RED ne répond jamais à un F.
✔ Le parsage est allégé (pas de gestion d'erreur complexe).
✔ Les modules F ont un traitement "fast lane" dans l'Arduino.

Cas de V – Catégorie utilisé pour les fonctions …

c'est **V** qui transporte :

- réglages,
- modes,
- activations,
- seuils,
- comportement,
- triggers,
- options,
- calibration,
- interactions entre modules.

Variables Stockage, abréviations & propriétés

Le nom utilisé pour la variable est son abréviation qui doit être unique .
Attention lors de la conception du fichier des variables globales de node-red.

Stockage Node-RED — 2 niveaux

A) runtime variables (global/flow/local) — pour traitement temps réel.

B) fichier `variables.json` — source de vérité pour initialisation + documentation.

Règles :

- À la réception d'un `$V:...#`, Node-RED **met à jour runtime (global)** puis, si la variable est verrouillable (`_locked=false`), écrit éventuellement dans `variables.json` (policy configurable : ecrire tout, jamais, ou périodiquement).
- Lors d'une modification manuelle via UI → modifier `variables.json` puis déclencher une repub SP→Arduino pour synchroniser.

Synchronisation initiale (P3.3 / P3.4)

But : initialiser Arduino avec valeurs stables du fichier variable Node-RED au démarrage SP.

Processus :

1. Au boot de SP / Node-RED, lire le fichier `variables.json` (ton fichier stable).
2. Construire un message `$V:...#` par variable/propriété/instance avec la **valeur initiale**.
3. Publier sur `arduino/commande` avec `urg=normal` (ou `urgent` si nécessaire).
    - Exemple topic SP → Arduino : `SP/V/normal/DVar` payload `$V:DVar:act:*:true#`
4. Arduino reçoit ces commandes et applique / ack via `arduino/reponse`: `$I:Sync:var:DVar:OK#` (ou `$E:Sync:var:DVar:ERR#`)
5. Node-RED consolide ack et met à jour son espace mémoire (global).

Pour SE la sync initiale est légère (Tasker déclenche l'envoi des quelques états initiaux au SP) — tu as dit Tasker gère la sync initiale.

Types & mapping

On a déjà cette information grace au fichier des variables coté Node-red. Donc on est pas obligé de la transporté dans le message. on s'en sert pour :

- Valider / caster les valeurs reçues avant de les stocker dans Node-RED (global/flow/local).
- Générer des variables Node-RED du type adapté (string, number, boolean, object, array).

Mapping recommandé :

- `string` → stocker en `string`
- `number` → la règle est que l'on utilise un parse entier.
- `boolean` → `true` / `false` (accept : "1","0","true","false")
- `object` → valeur JSON encodée (ex: `{"x":1,"y":2}`) — **dans le payload on transmet l'objet JSON minifié** (toujours <100 chars)
- `array` → JSON minifié
- `time` / `ts` → ISO8601 ou epoch (privilégier ISO en string)

Node-RED : à la réception, la function de parsing se base sur la `abbrVarG` pour retrouver `typeP` dans le fichier de variables et convertir la valeur.

INSTANCE

Représente un capteur précis ou *tous* les capteurs d'un module.

Formats possibles :

Forme	Signification
`*`	toutes les instances
`2`	instance 2
`[0,3]`	instances 0 et 3
`0-4` *(rare)*	plage d'instances (si activé)

Modules arduino

Architecture interne : "Modules indépendants + cœur système"

Chaque module est un **objet fonctionnel autonome**, placé dans son propre fichier :

- actuators/lrub.cpp
- actuators/lring.cpp
- actuators/Mt.cpp
- sensors/US.cpp
- sensors/Mic.cpp
- sensors/Odom.cpp
- sensors/Mvt_ir.cpp
- system/emergency.cpp
- system/SCfg.cpp
- etc.

Chaque module expose :

✔ initModule()

Initialisation

✔ processPeriodic()

Calculs périodiques internes (si applicable)

✔ read/write getters

Exemple :

- US_getDistance(i)
- Mic_getPeak()
- Odom_getPose(&x, &y, &theta)

✔ dispatch()

(le point de contact VPIV externe)

MODULE

Le module correspond au nom namespace (dispatch) dans Arduino.

Abréviation	Fichier concerné	Rôle
`lrub`	lrub.cpp	ruban LED
`lring`	lring.cpp	anneau LED
`Srv`	Srv.cpp	servos tête
`Mt`	Mt.cpp	moteurs via UNO
`US`	US.cpp	ultrasons
`Mic`	Mic.cpp	micro 3 canaux
`Odom`	Odom.cpp	encodeurs

Abréviation	Fichier concerné	Rôle
`Mvt_ir`	Mvt_ir.cpp	détecteur IR
`FS`	FS.cpp	force HX711
`SCfg`	SCfg.cpp	système / reset / emergency

Transfert d'info entre module

Transfert par lecture directe de fonctions (pull)

Le module A lit les données du module B à la demande :

Voici **l'explication détaillée, claire et complète** du fonctionnement des **transferts internes entre modules** dans ton architecture Arduino modulaire — c'est un point essentiel, car VPIV ne couvre que les échanges *externes*, alors que les interactions *internes* doivent être efficaces, non bloquantes, et propres.

Le module A lit les données du module B à la demande :

```
int d = US_getDistance(3);
```

```
float angle = Mic_getAngle();
```

```
Odom_getPose(&x, &y, &theta);
```

✔ Pas de copy inutile
✔ Basé sur getters
✔ Le module source garde ses calculs internes

Exemple concret :
Le module Odom récupère la vitesse réelle des roues dans Mt :

```
int leftSpeed  = Mt_getSpeedLeft();
```

```
int rightSpeed = Mt_getSpeedRight();
```

Transfert par buffer partagé (push)

Certains modules écrivent automatiquement dans un buffer accessible à d'autres.

Exemple :

- Mic calcule en continu `mic_current_peak` et `mic_direction`
- Ces valeurs sont stockées dans des variables globales protégées
- Un autre module (ex : comportement) lit ces valeurs quand il veut

```
extern volatile int mic_current_peak;
```

```
extern volatile float mic_direction;
```

✔ Idéal pour le flux haute fréquence de Mic & US
✔ Pas besoin d'appels croisés
✔ Pas de dépendance circulaire

Message interne (événement)

Très utile : un module peut “notifier” un événement à un autre via une file interne.

Exemple : si un sonar détecte un obstacle < seuil :

```
EventBus_push(EVENT_OBSTACLE, sonar_index);
```

Le module moteurs peut :

```
if(EventBus_has(EVENT_OBSTACLE)) {
    int idx = EventBus_pop();
    Mt_stop();
}
```

✔ Faible couplage
✔ Impossible de bloquer
✔ Extensible

Interaction complète : “commande externe → action interne → flux interne → info externe”

Un cycle complet ressemble à ceci :

1. Node-RED envoie :

```
$V:US:*:dang:20#
```

2. dispatch(US) transforme ceci en :

```
US_setDangerThreshold(20);
```

3. Quand un sonar détecte un obstacle :

```
if(US_distance[i] < US_danger_threshold) {
    EventBus_push(EVENT_OBSTACLE, i);
}
```

4. Le module moteur écoute :

```
if(EventBus_has(EVENT_OBSTACLE)) {
    Mt_stopAll();
    Emergency_raise("Obstacle");
}
```

5. Emergency déclenche :

- arrêt moteurs
- couleur LED rouge
- message externe :

`$E:Emergency:*:Alert:Obstacle#`

6. Node-RED ou Tasker peut envoyer :

`$V:SCfg:*:EMG:clear#`

Ceci retombe dans :

```
Emergency_clear();
```

Interaction interne complexe : “traitements secondaires”

Imagine :
Le sonar US déclenche une analyse micro pour déterminer direction d'évitement

Séquence interne :

1. US détecte obstacle → `EventBus_push(EVENT_OBSTACLE, idx)`
2. Un module “logic” (ou Mt lui-même) lit :

```
if(EventBus_has(EVENT_OBSTACLE)) {
    int idx = EventBus_pop();
    float dir = Mic_getDirection();
    Mt_rotateAwayFrom(dir);
}
```

✔ Aucun VPIV
✔ Aucun échange externe
✔ 100% interne, rapide et robuste

Où passent les paramètres “complexes / secondaires” ?

🔶 **Exemples de paramètres :**

- modes d’interaction multimodale (Mic + US)
- seuils adaptatifs
- facteurs de correction odométrie
- “mixage” capteurs pour décision
- activation d’un traitement secondaire interne

🔶 **La règle :**

✔ Ces paramètres sont reçus via **V**
✔ Stockés dans un module **pivot** (souvent SCfg, Odom, US, Mic)
✔ Les modules internes les lisent directement

Exemple :

```
$V:US:*:sec:mic#
```

Cela donne :

```
US_secondaryMode = SEC_MODE_MIC;
```

Ensuite dans US_processPeriodic :

```
if(US_secondaryMode == SEC_MODE_MIC) {
    float micDir = Mic_getDirection();
    // adaptation ou fusion
}
```

🎁 **Résumé**

Type d'échange interne	Usage	Modules concernés	Format
pull (getters)	Lecture de valeurs	Mt, US, Mic, Odom	fonction C++
push (buffers)	Mise à disposition continue	US, Mic, Odom	variables partagées
événements (EventBus)	Signaux, alertes, réactions immédiates	tous	queue interne

Tous les paramètres complexes viennent d'un VPIV V, mais sont utilisés en interne sans repasser par VPIV.

Propriété

Dépend du module.

Exemples :

Module	Propriétés courantes
LED	`rgb`, `int`, `blink`, `off`
US	`read`, `freq`, `dang`, `act`
Mic	`rms`, `peak`, `dir`, `raw`
Srv	`pos`, `spd`, `home`
Mt	`spd`, `rot`, `stop`
Odom	`reset`, `raw`, `pos`
FS	`raw`, `kg`, `tare`, `lim`
SCfg	`reset`, `test`, `mode`, `EMG:clear`

---

4 VALEUR

Dépend de la propriété.

Quelques exemples :

```
100                 ← entier
12.4                ← flottant
255,100,0           ← RGB
true / false        ← booléen
[1,3]               ← liste instances
```

Exemples de messages

Commande vers Arduino :

`$V:US:*:read:1#`

→ lit tous les ultrasons et répondra immédiatement via I ou F.

Retour d'un capteur :

`$I:US:2:dist:41#`

→ “sonar 2 détecte un obstacle à 41 cm”.

Erreur :

`$E:Mic:1:err:overflow#`

→ overflow du buffer audio.

Streaming haute fréquence :

`$F:Mic:*:rms:23#`

`$F:Odom:*:tick:1#`

Gestion des erreurs VPIV

Toutes les erreurs passent par :

`$E:<module>:<inst>:<prop>:<message>#`

Exemples :

Source	Message
Parsing	`bad_format`
Instance invalide	`bad_inst`
Valeur invalide	`bad_val`
Capteur déconnecté	`sensor_fail`
Urgence	`emergency`

Le module emergency utilise également :

`$I:EMG:*:code:<int>#`

Gestion des urgences

Définies dans `emergency.h` :

Code	Signification
0	aucune urgence
1	surchauffe
2	batterie faible
3	moteur bloqué
4	capteur défaillant
5	buffer overflow
6	parsing error

Communication :

Détection :

`$I:EMG:*:code:3#`

Réarmement via SCfg :

```
$V:SCfg:*:EMG:clear#
```

---

7. Flux périodique vs commandes

Mode	Type	Exemple	Unité
Commande ponctuelle	V	`$V:US:*:read:1#`	—
Streaming	F	`$F:Mic:*:rms:31#`	Hz
Info ponctuelle	I	`$I:Odom:*:pos:1.23,0.50#`	m

Chaque module possède un paramètre `freq` quand applicable.

---

8. Sécurité du protocole

Robustesse garantie par :

- délimiteurs `$` et #
- parsing strict
- emergency_stop() en cas d'erreur fatale
- erreurs clairement envoyées
- pas de buffer dynamique
- message maximum fixe (`VPIV_MAX_MSG_LEN 128`)

Pourquoi c'est mieux que l'ancien système tablette ?

Ancien :

❌ messages complexes
❌ dépendance JSON / Android
❌ besoin d'un parseur lourd
❌ messages non uniformes
❌ difficile à tester

Nouveau VPIV :

✔ **simple**
✔ **stable**
✔ **modules indépendants**
✔ **testable depuis Serial Monitor**
✔ **compatible MQTT**
✔ **aucune dépendance Android**
✔ **lisible en texte pur**

MODULE PAR MODULE (DOCUMENTATION COMPLÈTE)

Chaque module autonome de l'architecture Arduino Mega est présenté selon cette structure :

1. **Rôle / objectifs**
2. **Variables VPIV (module, instances, propriétés)**
3. **Structure interne (variables, algorithmes)**
4. **Exemples de commandes VPIV**
5. **Exemples de réponses**
6. **Notes, limites, extensions possibles**

l r u b (ruban LED RGB)

Fichier : `src/actuators/lrub.*`

1. Rôle

Contrôler un ruban LED RGB adressable (NeoPixel / WS2812 ou équivalent).
Utilisé pour l'expression dynamique, l'état émotionnel, les couleurs d'ambiance, ou les signaux d'alerte.

2. Variables VPIV

Élément	Valeur
Nom VPIV	`lrub`
Instances	`*` seulement (ruban = un seul objet)
Propriétés	`rgb`, `int`, `blink`, `off`, `anim`

Détail des propriétés :

Propriété	Type	Description
`rgb`	`R,G,B`	Définit la couleur fixe
`int`	entier 0–255	Intensité globale
`blink`	`R,G,B,T`	Clignotement couleur / période
`anim`	entier	Animation préprogrammée (0 = off)
`off`	—	Éteindre complètement

3. Structure interne

- Gestion via Adafruit NeoPixel ou FastLED
- Timeout possible : extinction automatique au bout d'un délai
- Fonction `lrub_processTimeout()` appelée par le `.ino`

4. Exemples VPIV

Allumer en rouge :

```
$V:lrub:*:rgb:255,0,0#
```

Baisser l'intensité :

```
$V:lrub:*:int:50#
```

Animation n°3 :

```
$V:lrub:*:anim:3#
```

5. Exemple de réponse

```
$I:lrub:*:rgb:255,0,0#
```

6. Limites

- Le ruban est gourmand en temps CPU si animations rapides
- Prévoir un système de réduction FPS si loop() dépasse 100 ms

---

l r i n g (anneau LED expressif)

Fichier : `src/actuators/lring.*`

1. Rôle

Anneau LED circulaire servant aux expressions “visuelles” du robot : humeur, états, réactions, etc.

2. Variables VPIV

Élément	Valeur
Nom VPIV	`lring`
Instances	`*`
Propriétés	`rgb`, `int`, `pulse`, `eye`, `off`

Propriétés :

Prop	Type	Description
`rgb`	`R,G,B`	couleur fixe
`pulse`	`R,G,B,T`	pulsation lumineuse
`eye`	`angle,int`	position d’un “œil” et intensité
`int`	0–255	intensité globale
`off`	—	extinction

3.

3.Structure interne

- Lib LED identique au ruban
- Gestion d’un pattern “œil” basé sur un angle
- Timeout via `lring_processTimeout()`

4. Exemples VPIV

Afficher un œil pointé à gauche :

```
$V:lring:*:eye:-45,200#
```

Couleur bleue :

```
$V:lring:*:rgb:0,0,255#
```

5. Réponse :

```
$I:lring:*:eye:-45#
```

6. Limites

- L’algorithme d’œil utilise une interpolation angulaire simple
- Peut être amélioré pour différentes formes d’œils (émotions)

S r v (servos tête / bouche)

Fichier : `src/actuators/Srv.*`

1. Rôle

Contrôle des servomoteurs de la tête du robot (pan/tilt, bouche articulée, etc.).

2. Variables VPIV

Élément	Valeur
Nom VPIV	`Srv`
Instances	`["AV","G","D"]` ou selon config
Propriétés	`pos`, `spd`, `act`, `home`, `raw`

Propriétés :

Prop	Type	Description
`pos`	0–180°	position angulaire
`spd`	0–255	vitesse logicielle (limite vitesse)
`act`	booléen	active/désactive le servo
`home`	—	va à la position de repos
`raw`	PWM µs	pour debug/maintenance

3. Structure interne

- Servo contrôlé via `Servo.h`
- Position interpolée selon la vitesse (`spd`)
- Table d'instances configurée dans `config.h`

4. Exemples VPIV

Incliner tête à droite :

```
$V:Srv:AV:pos:140#
```

Augmenter vitesse :

```
$V:Srv:*:spd:180#
```

5. Réponse

```
$I:Srv:AV:pos:140#
```

6. Limites

- Les servos consomment beaucoup → prévoir alimentation séparée
- La vitesse logicielle n'est pas garantie si loop() trop long

---

Mt (moteurs via Arduino UNO)

Fichier : `src/actuators/Mt.*`

1. Rôle

Ce module ne pilote **pas** les moteurs directement :
il **envoie des commandes** à l'Arduino UNO contrôleur moteur.

Le protocole UNO suit un format :

`<acc,spd,dir>`

2. Variables VPIV

Élément	Valeur
Nom VPIV	`Mt`
Instances	`["L","R"]` (gauche/droite)
Propriétés	`spd`, `acc`, `dir`, `stop`, `raw`

Propriétés :

Prop	Type	Description
`spd`	-255 à +255	vitesse signée
`acc`	0–255	accélération
`dir`	0/1	direction brute
`stop`	—	arrête moteur
`raw`	chaîne complète	envoie `<acc,spd,dir>` tel quel

3. Structure interne

- Communication série vers UNO : `MOTEURS_SERIAL`
- Buffer transmission protégé contre valeurs hors limites
- Fonction d'arrêt automatique en cas d'urgence

4. Exemples VPIV

Avancer doucement :

```
$V:Mt:L:spd:80#
$V:Mt:R:spd:80#
```

Stop :

```
$V:Mt:*:stop:1#
```

5. Réponse

```
$I:Mt:L:spd:80#
```

6. Limites

- Dépendance à la qualité du cable USB Mega ↔ UNO

- Certaines valeurs envoyées au UNO doivent être filtrées

U S (sonars ultrason HC-SR04)

Fichier : `src/sensors/US.*`

1. Rôle

Lecture précise des distances avec plusieurs sonars ultrasons HC-SR04.

2. Variables VPIV

Élément	Valeur
Nom VPIV	`US`
Instances	`[0..N-1]` selon config
Propriétés	`read`, `freq`, `dang`, `act`

Propriété	Type	Description
`read`	bool	demande de lecture immédiate
`freq`	ms	fréquence de scan périodique
`dang`	cm	seuil danger
`act`	bool	active/désactive capteur

3. Structure interne

- Lecture non bloquante (ping → wait → calcul)
- Timeout → valeur `-1` (capteur suspect)
- Moyenne interne sur plusieurs lectures
- Optimisation pour éviter de saturer MQTT :
  → transmission seulement si delta > seuil

4. Exemples VPIV

Lire tout :

```
$V:US:*:read:1#
```

Changer fréquence :

```
$V:US:*:freq:200#
```

Modifier seuil danger sonar 3 :

```
$V:US:3:dang:25#
```

5. Réponse

```
$I:US:3:dist:41#
```

Ou en cas de panne :

```
$E:US:3:err:timeout#
```

6. Limites

- Les ultrasons interfèrent entre eux → scans séquencés

- Fiabilité réduite sous 10 cm ou surfaces absorbantes

Mic (array de microphones + direction)

Fichier : `src/sensors/Mic.*`

1. Rôle du module

Le robot dispose de **trois microphones analogiques** (avant, gauche, droite).
L'objectif du module Mic est :

- Lire les intensités instantanées
- Détecter les variations significatives
- Estimer la **direction du son** (gauche/droite)
- Transmettre uniquement les variations utiles pour éviter le spam MQTT
- Servir de base au module *SoundTracker* (traitement avancé)

2. Variables VPIV

Élément	Valeur
Nom VPIV	`Mic`
Instances	`["AV","G","D"]`
Propriétés	`read`, `dir`, `act`, `th`, `mode`

Détails des propriétés

Prop	Type	Description
`read`	bool	lecture immédiate du micro
`dir`	lecture seule	renvoie direction du bruit
`act`	bool	active/désactive la mesure
`th`	0–100	seuil de variation (%)
`mode`	enum	`"raw"` ou `"dir"`

3. Structure interne

- Lecture analogique micro → conversion en %
- Filtrage par moyenne mobile
- Détection événementielle : variation > seuil
- Calcul direction du son :

```
dir = atan2( micG - micD , micAV );
```

Simplifié pour le projet en une estimation qualitative :

- `-1` = gauche
- `0` = face
- `+1` = droite

4. Exemples VPIV

Activer micro gauche :

`$V:Mic:G:act:1#`

Demander direction sonore :

`$V:Mic:*:dir:1#`

Modifier seuil :

`$V:Mic:*:th:20#`

5. Réponses

`$I:Mic:*:dir:-1#`

ou lecture brute :

`$I:Mic:G:read:42#`

6. Limites

- Sensibilité dépend de la position du robot et bruits du moteur
- Peut nécessiter calibration automatique
- Prochain module SoundTracker = amélioration du suivi dynamique

Odom (odométrie encodeurs)

Fichier : `src/sensors/Odom.*`

1. Rôle

Permet de calculer :

- la position du robot en X/Y
- son orientation (cap)
- sa distance parcourue
- l'angle relatif vers une cible éventuelle

Basé sur deux encodeurs incrémentaux (gauche / droite).

2. Variables VPIV

Élément	Valeur
Nom VPIV	`Odom`
Instances	`["X","Y","0"]` virtuelles
Propriétés	`act`, `pos`, `reset`, `raw`, `coef`

Propriété	Type	Description
`act`	bool	active / désactive l'odométrie
`pos`	lecture seule	renvoie X,Y,Θ
`reset`	—	remet à zéro
`raw`	—	renvoie les ticks bruts
`coef`	float	modifie calibration tick→mm

3. Structure interne

Fournit :

Mise à jour des ticks :

```
ΔG = tickG - oldTickG
ΔD = tickD - oldTickD
```

Distance :

```
d = (ΔG + ΔD) / 2 * coeff_mm
```

Orientation :

```
Δθ = (ΔD - ΔG) * coeff_deg
θ += Δθ
```

Position :

```
x += d * cos(θ)
y += d * sin(θ)
```

4. Exemples VPIV

Lire position :

`$V:Odom:*:pos:1#`

Reset :

`$V:Odom:*:reset:1#`

Changer coefficient (calibration) :

`$V:Odom:*:coef:0.0711#`

5. Réponses

Position :

`$I:Odom:*:pos:1023,540,12.4#`

Ticks bruts :

`$I:Odom:*:raw:4002,3995#`

6. Limites

- Les valeurs envoyées doivent être limitées en fréquence
- Les glissements de roues faussent les mesures
- Calibration indispensable pour précision

Mvt_ir (détecteur infrarouge PIR)

Fichier : `src/sensors/Mvt_ir.*`

1. Rôle

Détecter les mouvements humains/animaux dans l'environnement via un PIR.

2. Variables VPIV

Élément	Valeur
Nom VPIV	`Mvt_ir`
Instances	`*`
Propriétés	`act`, `read`, `mode`, `th`

Propriétés

Prop	Type	Description
`act`	bool	active capteur
`read`	bool	lecture directe
`mode`	`"pulse"` / `"hold"`	comportement
`th`	ms	durée min pour filtrage

3. Structure interne

- Lecture digitale simple
- Filtre anti-bruit (durée minimale)
- Modes :
  - **pulse** : événement unique
  - **hold** : 1 tant que mouvement persiste

4. Exemples VPIV

Activer PIR :

`$V:Mvt_ir:*:act:1#`

Lire valeur :

`$V:Mvt_ir:*:read:1#`

5. Réponse

```
$I:Mvt_ir:*:read:1#
```

6. Notes

- Sensible aux changements thermiques
- Le mode “pulse” est souvent préférable pour MQTT

FS (capteur de force HX711)

Fichier : `src/sensors/FS.*`

1. Rôle

Mesure le poids exercé sur la laisse du robot ou un capteur de traction.

2. Variables VPIV

Élément	Valeur
Nom VPIV	`FS`
Instances	`*`
Propriétés	`read`, `tare`, `cal`, `act`, `th`

Propriétés

Prop	Type	Description
`read`	bool	lecture immédiate
`tare`	—	remise à zéro
`cal`	float	calibration du gain
`act`	bool	active le module
`th`	int	seuil déclenchement événement

3. Structure interne

- Utilise la librairie HX711
- Moyenne sur N lectures pour stabilité
- Déclenchement événement uniquement si variation significative

4. Exemples VPIV

Lire poids :

```
$V:FS:*:read:1#
```

Tare :

```
$V:FS:*:tare:1#
```

Changer calibration :

```
$V:FS:*:cal:5895655#
```

5. Réponses

```
$I:FS:*:read:4.32#
```

6. Limites

- Le HX711 est sensible aux vibrations
- Prévoir un filtrage si robot en mouvement

SCfg (Système / Réglages / Emergency)

Fichier : `src/system/SCfg.*`

1. Rôle général du module SCfg

Module *central* regroupant :

- Gestion des modes du robot
- Reset logiciel / réinitialisation modules
- Gestion des états d'urgence
- Activation du mode test
- Configuration générale (timings, debug, etc.)

2. Variables VPIV

Élément	Valeur
Nom VPIV	`SCfg`
Instances	`*`
Propriétés	`mode`, `reset`, `emg`, `test`, `dbg`

Détails :

Prop	Type	Description
`mode`	enum	ARRET / VEILLE / SURVEILLANCE / AUTONOME / SUIVEUR / TELECOMMANDE
`reset`	—	reset général
`emg`	`"clear"` / `"state"`	contrôle urgence
`test`	bool	active le mode test général
`dbg`	bool	verbose modules

3. Rôle dans la chaîne d'urgence

Le module contrôle le flag global :

`emergency_active`

Si activé :

- tous les modules doivent refuser leurs commandes
- sauf `SCfg:emg:clear`

4. Exemples VPIV

Passer en mode autonome :

`$V:SCfg:*:mode:3#`

Activer état d'urgence (ex : traction laisse) :

`$V:SCfg:*:emg:1#`

Effacer l'urgence manuellement :

`$V:SCfg:*:emg:clear#`

Reset total :

`$V:SCfg:*:reset:1#`

5. Réponses

`$I:SCfg:*:mode:3#`

ou état d'urgence :

`$I:SCfg:*:emg:1#`

6. Notes

- Ce module doit rester minimaliste mais robuste
- **Peut intégrer les tests internes du robot (autotests)**

Parfait — je génère exactement le tableau synthétique dont tu as besoin pour Node-RED, module par module, avec :

✔ Variables internes Arduino
✔ Propriétés VPIV
✔ Instances utilisables
✔ Constantes (depuis `config.h`)
✔ Valeur / rôle
✔ Nom EXACT utilisé dans le dispatch

🎯 **Ce tableau est conçu spécialement pour l'écran d'initialisation de Node-RED**, afin que ton flow puisse envoyer automatiquement les valeurs initiales.

✅ TABLEAU GLOBAL VPIV — Tous les modules (prêt pour Node-RED)

(Version concise mais complète — je peux aussi générer une version XXL si tu veux la mettre en documentation)

🟦 1 — Module LRUB (Ruban LED)

Module VPIV : `lrub`

Élément	Nom VPIV	Type	Valeurs	Description
Instance	`*` ou liste indices `[0,5,7]`	indices	0–N	Pixel(s) à modifier
Prop couleur	`C`	prop	`r,g,b`	Applique une couleur
Prop intensité	`int`	prop	0–255	Brightness du ruban
Prop active	`act`	prop	0/1	Allumer / éteindre
Prop effet	`fx`	prop	`fuite`	Effet spécial interne
Timeout	`tout`	prop	ms	Durée avant extinction

Constantes importantes (`config.h`)

Constante	Valeur	Rôle
`RUBAN_LED_PIN`	`xx`	Pin NeoPixel
`RUBAN_NUM_PIXELS`	`xx`	Nombre de LEDs
`rubanTpsFin`	var	Timeout interne

🟦 2 — Module LRING (Anneau LED expressif)

Module VPIV : `lring`

Élément	Nom	Type	Valeurs
Instance	`*`	inst	idem
Couleur	`C`	prop	`r,g,b`
Intensité	`int`	prop	0–255
Expression	`fx`	prop	`sourire`, `alerte`
Timeout	`tout`	prop	ms

Constantes

Constante	Rôle
`LRING_NUM_PIXELS`	nb LEDs
`ringTpsFin`	timer

🟦 3 — Module SERVOS

Module VPIV : `Srv`

Élément	VPIV	Valeurs	Description
Instance	`0`, `1`…	servos individuels	tête, bouche, etc
Position	`pos`	0–180	angle
Vitesse	`spd`	1–10	vitesse interne
Active	`act`	0/1	Activer/Désactiver

Constantes

Constante	Rôle
`SERVO_PIN_X`	pin servo
`SERVO_POS_DEFAULT`	pos init

🟦 4 — Module MOTEURS (UNO)

Module VPIV : `Mt`

Élément	VPIV	Valeur
Accélération	`acc`	0–9
Vitesse	`spd`	0–9
Direction	`dir`	0–9
Stop	`stop`	1

Constantes

Constante	Rôle
`transMoteurP[]`	trame envoyée
`MOTEURS_SERIAL`	port série UNO

🟦 5 — Module ULTRASON (US)

Module VPIV : `Us`

Élément	VPIV	Type	Valeurs
Instances	`*`, `0`, `[0,2]`	indexes	sonar indexes
Lecture	`read`	prop	1
Fréquence	`freq`	prop	ms
Delta	`delta`	prop	cm
Danger	`dang`	prop	cm
SendAll	`all`	prop	0/1
Actif	`act`	prop	0/1

Constantes

Constante	Rôle
`SONAR_NUM`	nombre de capteurs
`US_PING_CYCLE_MS`	cycle global
`US_DELTA_THRESHOLD`	delta

Constante	Rôle
`sonarDanger[]`	seuil par sonar

🟦 6 — Module MICROPHONES (Mic)

Module VPIV : `Mic`

Élément	VPIV	Valeurs
Instance	`*` ou index	0–2
Lecture	`read`	raw / rms / peak / orient
Fenêtre	`win`	ms
Fréquence	`freq`	ms
Seuil	`th`	0–1023
Mode actif	`act`	0/1
Retour auto (alert)	`alert`	>seuil

Constantes

Constante	Rôle
`MIC_COUNT=3`	nombre micro
`mic_win_ms`	fenêtre RMS
`mic_freq_ms`	rythme lecture
`micThreshold[]`	seuils

🟦 7 — Module ODOMÉTRIE (Odom)

Module VPIV : `Odom`

Élément	VPIV	Valeurs
Reset	`reset`	1
Lecture	`read`	pos/angle/distance
Mode	`mode`	simple / ext

Constantes

Constante	Rôle
`ENC_LEFT_PIN_A` etc	encodeurs
`TICKS_PER_REV`	résolution

🟦 8 — Module IR (MvtIR)

Module VPIV : `MvtIR`

Élément	Prop	Valeurs
Lecture	`read`	1
Actif	`act`	0/1

Élément	Prop	Valeurs
Fréquence	`freq`	ms
Seuil alerte	`thd`	nombre détections
Mode	`mode`	idle / monitor / surveillance

Constantes

Constante	Rôle
`MVTIR_FREQ_DEFAULT`	freq
`MVTIR_THD_DEFAULT`	seuil
pin IR	hardware

🟦 9 — Module FS (Capteur de force)

Module VPIV : `FS`

Élément	Prop	Valeurs
Lecture	`read`	raw/value
Tare	`tare`	1
Calibration	`cal`	float
Seuil	`thd`	float
Fréquence	`freq`	ms
Mode	`mode`	idle / monitor / surveillance
Actif	`act`	0/1

Constantes

Constante	Rôle
`FS_CAL_DEFAULT`	facteur
`FS_FREQ_DEFAULT`	fréquence
`FS_THRESHOLD_DEFAULT`	seuil

🟦 10 — Module SYSTÈME (SCfg)

Module VPIV : `SCfg`

Élément	Prop	Valeurs	Description
Reset	`reset`	1	Reset Arduino
Emergency clear	`emg`	clear	sortie d'urgence
Mode test	`mode`	test/normal	active tests internes
Version	`ver`	1	demande version

Constantes

Constante	Rôle
`emergencyStop`	flag global
timers systèmes	comportement

🟧 RÉCAP POUR NODE-RED (clé → valeur)

Exemple structure d'init Node-RED :

```
{
  "lrub": {
    "int": 120,
    "act": 1
  },
  "lring": {
    "int": 80,
    "fx": "none"
  },
  "Mic": {
    "freq": 100,
    "win": 20,
    "th": 40
  },
  "Us": {
    "freq": 80,
    "delta": 10,
    "dang": [20,20,20,20],
    "all": 0
  },
  "FS": {
    "freq": 200,
    "thd": 50,
    "cal": 1.0
  },
  "MvtIR": {
    "freq": 150,
    "thd": 1,
    "mode": "monitor"
  }
}
```

VI — ARCHITECTURE DE TEST DU ROBOT

L'objectif : avoir un **mode test complet**, lancé via VPIV :

```
$V:SCfg:*:test:1#
```

Le test doit :

- vérifier chaque module
- envoyer un rapport détaillé vers Node-RED
- se terminer automatiquement

---

1 — Types de tests intégrés

Module	Test
lrub	allumer chaque LED 1 par 1
lring	sourire / triste / alerte
Srv	rotation min / max puis centrage
Mt	test vitesse lente / rapide
US	mesure distance + comparaison min/max
Mic	bruit test → direction
Odom	déplacement 10 cm → vérification ticks
Mvt_ir	détection événement
FS	mesure poids + tare
SCfg	test emergency / modes

---

2 — Format des rapports

Exemple :

```
$I:Test:Srv:result:OK#
```

```
$I:Test:Mic:result:left_detected#
```

```
$I:Test:FS:result:nominal#
```

```
$I:Test:Mt:result:fail_low_power#
```

---

3 — Tests automatisés en Node-RED

Création d'un dashboard :

- Bouton "Lancer test complet"
- Journal affiché ligne par ligne

- Envoi automatique de `$V:SCfg:*:test:1#`

PERSPECTIVES ET EXTENSIONS - A FAIRE

----

1 — Ajout d’une boussole externe (I2C QMC5883L)

Permet :

- orientation absolue
- calibration automatique
- fusion avec odométrie

---

2 — Fusion capteurs pour autonomie

Combinaison :

- Mic (direction du son)
- US (obstacles)
- IR (mouvement)
- Odométrie
- servos pour recherche de sources sonores

---

3 — Calibration assistée via VPIV

Exemples :

```
$V:Odom:*:cal_req:1#
$V:US:*:cal:save#
```

---

4 — Logs experts

Streaming haute fréquence :

```
$S:Mic:raw:12,34,21#
$S:Odom:raw:4021,3999#
$S:US:raw:78#
```

ANNEXES

Tableau (à faire)

Variable- Abrév clef - Module Arduino – Propriété – Pin associé